

ANÁLISIS DEL RETO

William Pollock, 202221321, w.pollock@uniandes.edu.co

Juan Alejandro Hernández, 202225518, ja.hernandezg12@uniandes.edu.co

Juan Manuel Bonilla, 202121983, j.bonillac@uniandes.edu.co

Requerimiento 1

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_1(data_structs, initialDate, finalDate):
    """
    Función que soluciona el requerimiento 1
    """
    lst = om.values(data_structs["times"], initialDate, finalDate)
    x = lt.newList("ARRAY_LIST")
    for cada in lt.iterator(lst):
        for elements in lt.iterator(cada["sismos"]):
            lt.addLast(x, elements)
    a = [lt.getElement(x,lt.size(x)),
         lt.getElement(x,lt.size(x)-1),
         lt.getElement(x,lt.size(x)-2),
         lt.getElement(x,3),
         lt.getElement(x,2),
         lt.getElement(x,1)]
    head = ["mag", "lat", "long", "depth", "sig", "gap", "nst", "title", "cdi", "mmi", "magType", "type", "code"]
    k = []
    for seis in a:
        l = {}
        for header in head:
            l[header] = seis[header]

        m = tabulate([l], headers="keys", tablefmt="grid")
        date = seis["time"]
        events = 1
        lista = [date, events, m]
        k.append(lista)

    t = tabulate(k, headers=["time","events","details"], tablefmt="grid")

    return t
```

Este requerimiento se encarga de retornar la cantidad de eventos sísmicos entre la fecha inicial y final insertada por el usuario. Lo primero que hace es escoger todos los valores en la estructura de datos que estén entre la fecha inicial y final. Después, crea una lista vacía y se usa 'getElement' para agregar y conseguir los primeros y últimos tres elementos que cumplen con la entrada. Finalmente, se escogen ciertos títulos de los archivos para hacer tabulate la información.

Entrada	Estructuras de datos del modelo, fecha inicial, fecha final.
Salidas	Eventos sísmicos entre la fecha inicial y final.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar valores en rango de fechas (values)	$O(N)$
Crear lista vacía (newList)	$O(1)$
Ciclo "sismos"	$O(N^2)$
Buscar primeros 3 y últimos 3 elementos(getElement)	$O(1)$
Título para tabulate (head)	$O(1)$
TOTAL	$O(N^2)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bits

Entrada	Tiempo (ms)
small	11125
5 pct	12735
10 pct	14764
20 pct	13701
30 pct	15166
50 pct	18428
80 pct	20285
large	20460

Tablas de datos

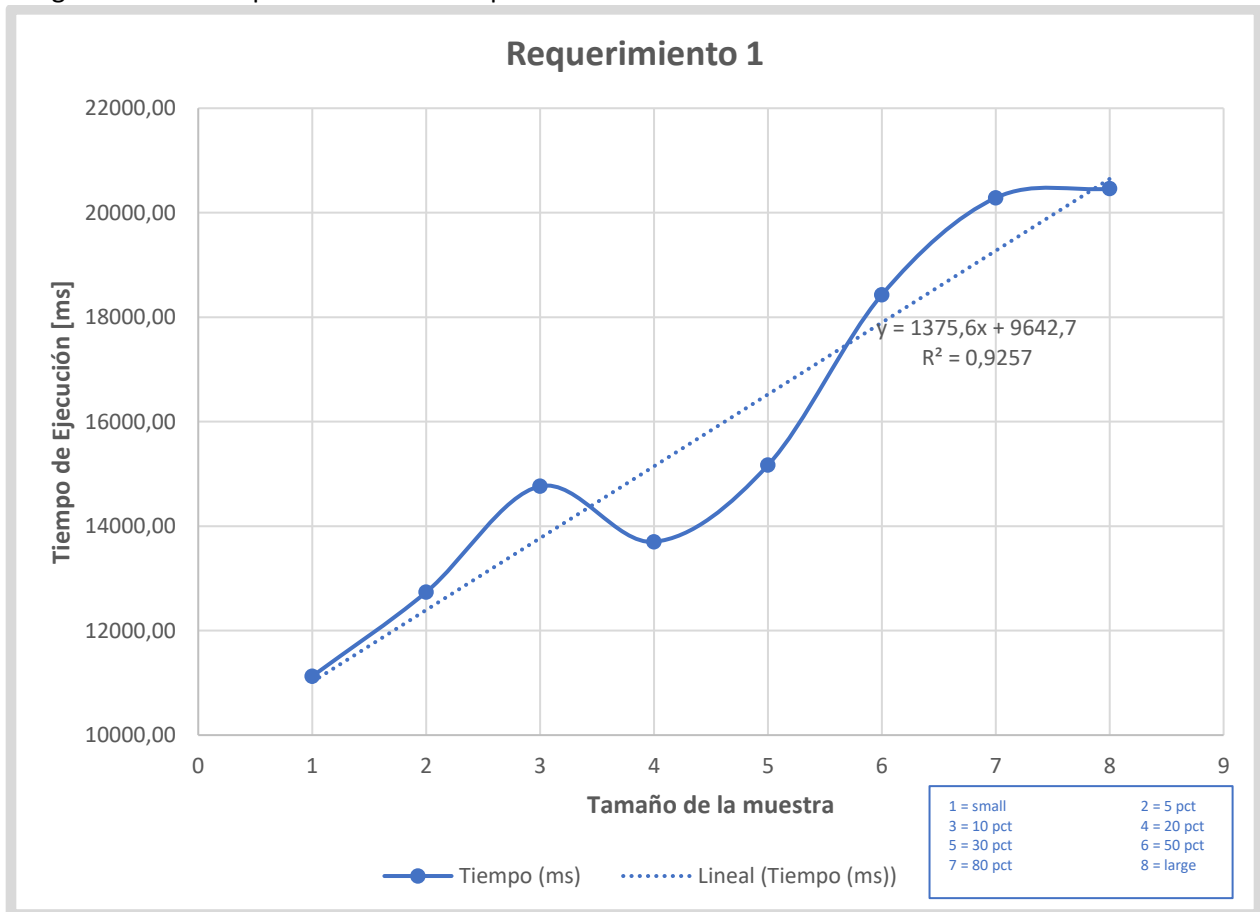
Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	11125
5 pct	Dato2	12735
10 pct	Dato3	14764
20 pct	Dato4	13701
30 pct	Dato5	15166

50 pct	Dato6	18428
80 pct	Dato7	20285
large	Dato8	20460

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

A pesar de que la función tuviera un ciclo dentro de otro (lo cual usualmente tiene una complejidad de $O(N^2)$), parece que tiene una complejidad de $O(N)$. Esto probablemente ocurre debido a que el primer ciclo (for seis in a) está usando la lista “a”, la cual tiene una complejidad de $O(1)$ mientras que el segundo ciclo usa “head”, otra lista que también tiene la misma complejidad. Estas dos resultan en $O(1*1) = O(1)$. En conclusión, la función parece tener una complejidad lineal.

Algo que nos resultó extraño sobre las pruebas fue el hecho de que el 20% de los datos cargara más rápido que el 10%. Volvimos a hacer las pruebas múltiples veces para verificar que esto estuviera bien y los resultados no mostraron lo contrario en ninguna prueba.

Requerimiento 2

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_2(data_structs, mag_ini, mag_fin):
    """
    Función que soluciona el requerimiento 2
    """
    lst = om.values(data_structs["magnitudes"], mag_ini, mag_fin)

    a = [lt.getElement(lst,lt.size(lst)),
          lt.getElement(lst,lt.size(lst)-1),
          lt.getElement(lst,lt.size(lst)-2),
          lt.getElement(lst,3),
          lt.getElement(lst,2),
          lt.getElement(lst,1)]
    head = ["time", "lat", "long", "depth", "sig", "gap", "nst", "title", "cdi", "mmi", "magType", "type", "code"]
    k = []
    for elementos in a:
        x = elementos["sismos"]
        if lt.size(x) > 6:
            z = lt.newList("ARRAY_LIST")
            o = quk.sort(x, compare_dates)
            lt.addLast(z,lt.getElement(o,lt.size(o)))
            lt.addLast(z,lt.getElement(o,lt.size(o)-1))
            lt.addLast(z,lt.getElement(o,lt.size(o)-2))
            lt.addLast(z,lt.getElement(o,3))
            lt.addLast(z,lt.getElement(o,2))
            lt.addLast(z,lt.getElement(o,1))
            x = z
        t = []
        for cada_uno in lt.iterator(x):
            l = []
            for header in head:
                l.append(cada_uno[header])
            t.append(l)
        mag = lt.getElement(x, 1)["mag"]
        events = lt.size(x)
        m = tabulate(t, headers=head, tablefmt="grid")
        lista = [mag, events, m]
        k.append(lista)

    t = tabulate(k, headers=["mag","events","details"], tablefmt="grid")
    return t
```

Este requerimiento se encarga de retornar los eventos sísmicos entre dos magnitudes escogidas por el usuario. Lo primero que hace es escoger todos los valores en la estructura de datos que estén entre la magnitud inicial y final. Después, se usa 'getElement' para conseguir los primeros 3 y últimos 3 valores encontrados dentro del rango. Dos listas vacías son creadas (k & z). En la segunda los elementos son ordenados siempre y cuando tengan un tamaño menor a 6. Además, se crea otras dos listas vacías otra vez (t & l), la segunda entrando en la primera después de que contenga valores de "head". Finalmente, la lista "x" se usa para extraer los valores de magnitud, los cuales son usados en "k" (la cual es usada en el tabulate de la función).

Entrada	Estructura de datos del modelo, magnitud inicial, magnitud final.
Salidas	Eventos sísmicos entre el rango de magnitudes.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar valores en rango de magnitudes	O(N)

Creación lista "a"	$O(1)$
"head"	$O(1)$
Creación lista "k"	$O(1)$
Ciclo "for elementos in a"	$O(1)$
Condicional "if $lt.size(x) > 6$ "	$O(1)$
Ordenamiento de datos	$O(N \log N)$
Ciclo "iterator"	$O(N)$
Append de lista k	$O(1)$
TOTAL	$O(N \log N)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bits

Entrada	Tiempo (ms)
small	3290
5 pct	3630
10 pct	3797
20 pct	4561
30 pct	5244
50 pct	6106
80 pct	6934
large	8248

Tablas de datos

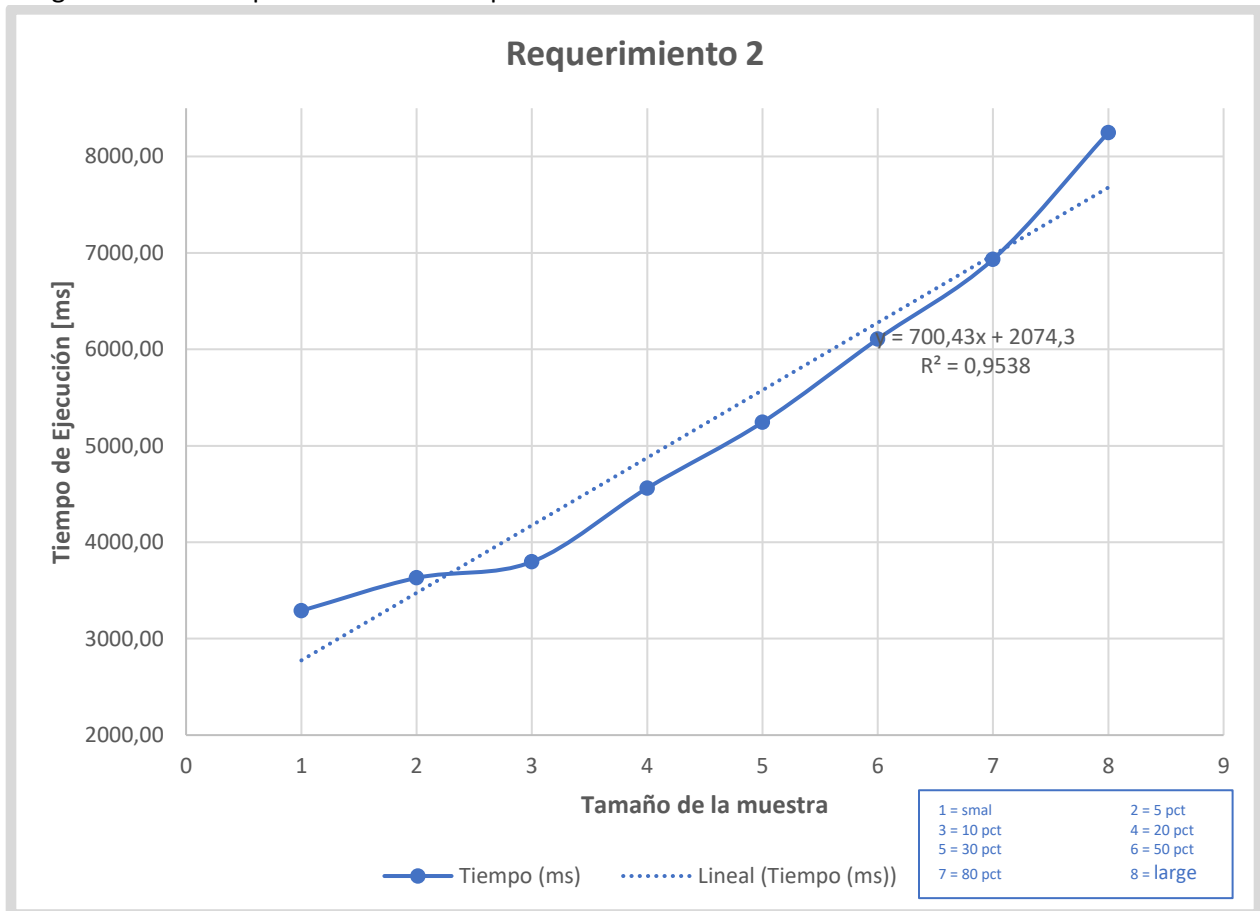
Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	3290
5 pct	Dato2	3630
10 pct	Dato3	3797
20 pct	Dato4	4561
30 pct	Dato5	5244
50 pct	Dato6	6106

80 pct	Dato7	6934
large	Dato8	8248

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Parece que hay concordancia entre la gráfica y la complejidad predecida, ya que la primera parece tener forma de $O(N \log N)$. También pensamos esto debido a que los últimos valores (desde '30 pct' a 'large') comenzaron a demorarse más tiempo en cargar, por lo cual la gráfica tiene un aspecto encurvado. En conclusión, parece que el ordenamiento de datos ("qk.sort") fue la parte de la función con mayor complejidad, ya que los ciclos tuvieron una complejidad de $O(1 \cdot N) = O(N)$.

Requerimiento 3

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_3(data_structs,magnitud,profundidad):
    """
    Función que soluciona el requerimiento 3
    """
    # TODO: Realizar el requerimiento 3
    listabrutos = om.values(data_structs["magnitudes"],magnitud, om.maxKey(data_structs["magnitudes"]))
    listadatos = lt.newList("ARRAY_LIST")
    for i in lt.iterator(listabrutos):
        for j in lt.iterator(i["sismos"]):
            lt.addLast(listadatos,j)
    prof = om.newMap(omaptype="RBT")
    for k in lt.iterator(listadatos):
        if k["depth"] != "":
            updateProfundidadIndex(prof, k)
    listabrutos = om.values(prof, om.minKey(prof), profundidad)
    listadatos = lt.newList("ARRAY_LIST")
    for l in lt.iterator(listabrutos):
        for m in lt.iterator(l["sismos"]):
            lt.addLast(listadatos, m)
    lenght1 = lt.size(listadatos)
    listaorganizar= quk.sort(listadatos, compare_dates)
    #Creacion Sublista primeros 10 Eventos
    listaprint = [lt.getElement(listaorganizar,lt.size(listaorganizar)),
                  lt.getElement(listaorganizar,lt.size(listaorganizar)-1),
                  lt.getElement(listaorganizar,lt.size(listaorganizar)-2),
                  lt.getElement(listaorganizar,lt.size(listaorganizar)-7),
                  lt.getElement(listaorganizar,lt.size(listaorganizar)-8),
                  lt.getElement(listaorganizar,lt.size(listaorganizar)-9)]
    #Generacion tabulate
    head = ["mag", "lat", "long", "depth", "sig", "gap", "nst", "title", "cdi", "mmi", "magType", "type", "code"]
    n = []
    for f in listaprint:
        l = {}
        for header in head:
            l[header] = f[header]

        m = tabulate([l], headers="keys", tablefmt="grid")
        date = f["time"]
        events = 1
        lista = [date, events, m]
        n.append(lista)

    t = tabulate(n, headers=["time","events","details"], tablefmt="grid")
    return t,lenght1
```

Este requerimiento se encarga de conseguir los 10 eventos más recientes según una magnitud mínima y una profundidad máxima escogidas por el usuario. Lo primero que hace es buscar valores en el árbol de magnitud y después meterlos en una lista vacía. Con esto, crea un nuevo mapa con tabla de hash para conseguir los valores de profundidad. Después de haber buscado estos valores, la lista se organiza por fecha y se crea otra lista para imprimir los resultados. Finalmente, se usa tabulate para que cumpla el formato requerido.

Entrada	Estructura de datos del modelo, magnitud, profundidad.
Salidas	Los 10 eventos más recientes según una magnitud mínima y profundidad máxima escogidas por el usuario.
Implementado (Sí/No)	Sí. Juan Manuel Bonilla.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Busca Valores en Árbol Magnitud (Rango)	$O(\log N)$
Meter en lista valores en el rango de magnitud	$O(\log N)$
Crea nuevo mapa con tabla de hash de profundidad	$O(1)$
Busca Valores en Árbol Profundidad (Rango)	$O(\log N)$
Organizar la lista por fecha	$O(\log N)$
Creación lista para imprimir	$O(1)$
Código para Tabulate	$O(\log N)$
TOTAL	$O(\log N)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bits

Entrada	Tiempo (ms)
small	6012
5 pct	6035
10 pct	6054
20 pct	7569
30 pct	8361
50 pct	9745
80 pct	14869
large	18377

Tablas de datos

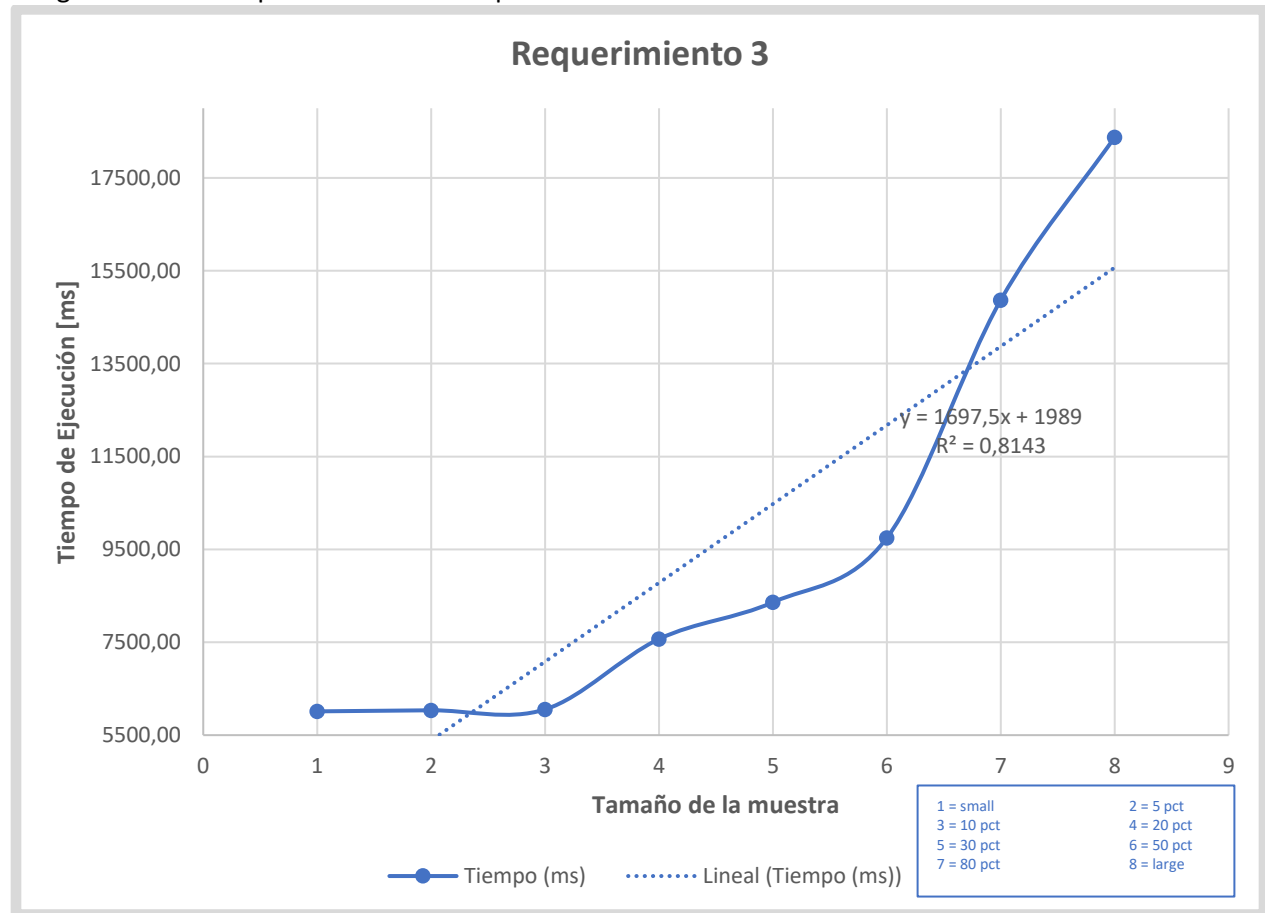
Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	6012
5 pct	Dato2	6035
10 pct	Dato3	6054

20 pct	Dato4	7569
30 pct	Dato5	8361
50 pct	Dato6	9745
80 pct	Dato7	14869
large	Dato8	18377

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Después de ver la gráfica y la tabla de complejidad, parece que sobreestimamos la organización de la lista por fechas. En este caso usamos quicksort, el cual puede tener una complejidad de $O(\log N)$ en sus mejores casos; sin embargo, parece que en este requerimiento tuvo una complejidad de $O(N \log N)$, el cual es el caso promedio de este tipo de ordenamiento.

Requerimiento 4

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_4(data_structs, sig, distancia):  
    """  
    Función que soluciona el requerimiento 4  
    """  
    lst = om.values(data_structs["significance"], sig, om.maxKey(data_structs["significance"]))  
    x = lt.newList("ARRAY_LIST")  
    for cada in lt.iterator(lst):  
        for elements in lt.iterator(cada["sismos"]):  
            lt.addLast(x, elements)  
    az = om.newMap(omaptype="RBT")  
    for cu in lt.iterator(x):  
        if cu["gap"] != "":  
            updateDistanceIndex(az, cu)  
    lst = om.values(az, om.minKey(az), distancia)  
    x = lt.newList("ARRAY_LIST")  
    for cada in lt.iterator(lst):  
        for elements in lt.iterator(cada["sismos"]):  
            lt.addLast(x, elements)  
    o = quk.sort(x, compare_dates)  
    a = [lt.getElement(o,lt.size(o)),  
        lt.getElement(o,lt.size(o)-1),  
        lt.getElement(o,lt.size(o)-2),  
        lt.getElement(o,3),  
        lt.getElement(o,2),  
        lt.getElement(o,1)]  
    head = ["mag", "lat", "long", "depth", "sig", "gap", "nst", "title", "cdi", "mmi", "magType", "type", "code"]  
    k = []  
    for seis in a:  
        l = {}  
        for header in head:  
            l[header] = seis[header]  
        m = tabulate([l], headers="keys", tablefmt="grid")  
        date = seis["time"]  
        events = 1  
        lista = [date, events, m]  
        k.append(lista)  
    t = tabulate(k, headers=["time","events","details"], tablefmt="grid")  
    return t
```

Este requerimiento se encarga de conseguir los 15 eventos sísmicos más recientes según una significancia mínima y una distancia azimutal máxima escogidas por el usuario. Lo primero que hace es buscar valores en el árbol de significancia y meterlos en una lista vacía. Después, crea un nuevo mapa para conseguir los valores de las distancias azimutales, los cuales son ordenados para que el usuario reciba los tres primeros y tres últimos resultados. Finalmente, se crean los títulos para el tabulate del resultado.

Entrada	Estructura de datos del modelo, significancia, distancia azimutal.
Salidas	Los 15 eventos sísmicos más recientes según una significancia mínima y distancia azimutal máxima escogidas por el usuario.
Implementado (Sí/No)	Sí. Juan Alejandro Hernández.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar valores en el árbol de significancia	$O(\log N)$
Insertar en lista valores rango significancia	$O(\log N)$
Crear nuevo mapa para distancias azimutales	$O(1)$
Buscar valores en el árbol de distancias azimutales	$O(\log N)$
Organizar lista (quick sort)	$O(\log N)$
Creación títulos para tabulate	$O(1)$
Código para tabulate	$O(\log N)$
TOTAL	$O(\log N)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bitsd

Entrada	Tiempo (ms)
small	3177
5 pct	3407
10 pct	3604
20 pct	4363
30 pct	5173
50 pct	7425
80 pct	9359
large	10658

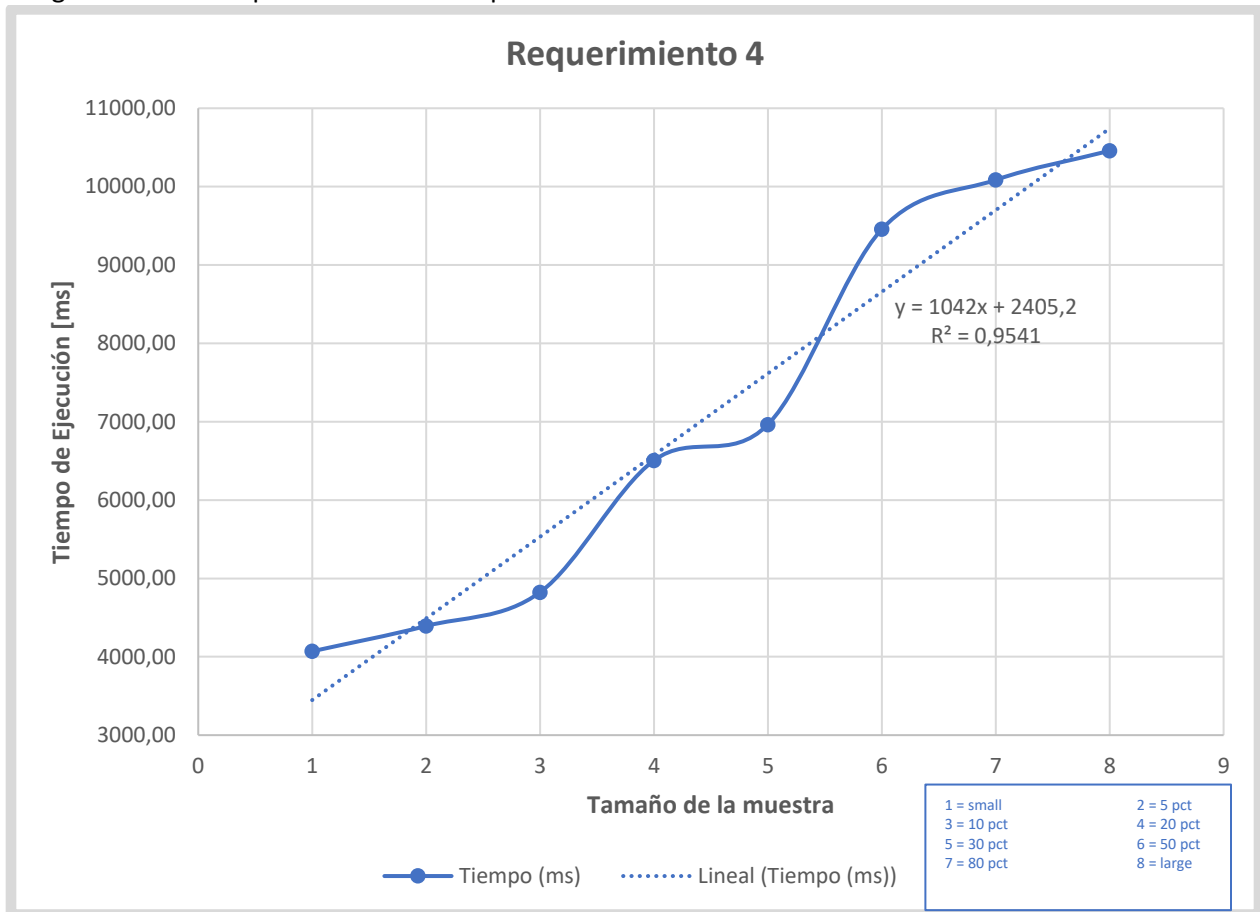
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	3177
5 pct	Dato2	3407
10 pct	Dato3	3604
20 pct	Dato4	4363
30 pct	Dato5	5173
50 pct	Dato6	7425
80 pct	Dato7	9359
large	Dato8	10658

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Esta gráfica parece estar entre $O(N)$ y $O(\log N)$, pero sentimos que está más cerca a la primera debido a que los últimos valores de la carga se comenzaron a demorar más tiempo en cargar. Por ejemplo, en la tabla de datos se puede ver que los primeros tres valores seguían dentro de los 3000 milisegundos mientras que desde la carga del 30% y la del 80% hubo un incremento de 4000 milisegundos.

Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_5(data_structs, profundidad, estacion):  
    """  
    Función que soluciona el requerimiento 5  
    """  
    # TODO: Realizar el requerimiento 5  
    lst = om.values(data_structs["estacion"], str(estacion), om.minKey(data_structs["estacion"]))  
    lst2 = lt.newList("ARRAY_LIST")  
    for i in lt.iterator(lst):  
        for j in lt.iterator(i["sismos"]):  
            lt.addLast(lst2, j)  
    prof = om.newMap(omapttype="RBT")  
    for k in lt.iterator(lst2):  
        if k["depth"] != "":  
            updateProfundidadIndex(prof, k)  
    lst = om.values(prof, om.maxKey(prof), profundidad)  
    lst2 = lt.newList("ARRAY_LIST")  
    for l in lt.iterator(lst2):  
        for m in lt.iterator(l["sismos"]):  
            lt.addLast(lst2, m)  
    tamaño = lt.size(lst2)  
    sorteo = quk.sort(lst2, compare_dates)  
    lprint = [lt.getElement(sorteo, lt.size(sorteo)),  
              lt.getElement(sorteo, lt.size(sorteo)-1),  
              lt.getElement(sorteo, lt.size(sorteo)-2),  
              lt.getElement(sorteo, lt.size(sorteo)-17),  
              lt.getElement(sorteo, lt.size(sorteo)-18),  
              lt.getElement(sorteo, lt.size(sorteo)-19)]  
    head = ["mag", "lat", "long", "depth", "sig", "gap", "nst", "title", "cdi", "mmi", "magType", "type", "code"]  
    w = []  
    for x in lprint:  
        y = {}  
        for header in head:  
            y[header] = x[header]  
        z = tabulate([1], headers="keys", tablefmt="grid")  
        date = x["time"]  
        events = 1  
        lista = [date, events, z]  
        w.append(lista)  
    tab = tabulate(w, headers=["time", "events", "details"], tablefmt="grid")  
    return tab, tamaño
```

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Estructura de datos del modelo, profundidad, estación.
Salidas	Los 20 eventos más recientes para una profundidad mínima y una cantidad de estaciones máxima escogidas por el usuario.
Implementado (Sí/No)	Sí. William Pollock

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1	$O(\dots)$
Paso 2	$O(\dots)$
Paso	$O(\dots)$
TOTAL	$O(\dots)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bits

Entrada	Tiempo (s)

Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Graficas

Las gráficas con la representación de las pruebas realizadas.

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Requerimiento 6

Requerimiento 7

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_7(data_structs, año, titulo, propiedad, bins):  
    """  
    Función que soluciona el requerimiento 7  
    """  
    x = lt.newList("ARRAY_LIST")  
    a = me.getValue(mp.get(data_structs["fechas"], año))  
    for cu in lt.iterator(a["sismos"]):  
        if titulo in cu["title"]:  
            lt.addLast(x, cu)  
    datos = []  
    for todos in lt.iterator(x):  
        datos.append(todos[propiedad])  
    datos.sort()  
    x = quk.sort(x, compare_dates)  
    o = [lt.getElement(x, lt.size(x)),  
        lt.getElement(x, lt.size(x)-1),  
        lt.getElement(x, lt.size(x)-2),  
        lt.getElement(x, 3),  
        lt.getElement(x, 2),  
        lt.getElement(x, 1)]  
    k = []  
    head = ["time", "lat", "long", "title", "code", "mag"]  
    for seis in o:  
        l = {}  
        for header in head:  
            l[header] = seis[header]  
        k.append(l)  
    t = tabulate(k, headers="keys", tablefmt="grid")  
    return (datos, t)
```

Este requerimiento se encarga de graficar un histograma basado en el año, región, propiedad (magnitud, profundidad, significancia) y cantidad de casillas escogidas por el usuario. Lo primero que se hizo fue crear una lista vacía y agregarle los valores del árbol de fechas. Después, se crea otra lista y a esta se le agregan los valores anteriores, los cuales son organizados en términos tamaño. Finalmente, se crean los títulos para el tabulate.

Entrada	Estructura de datos del modelo, año, título de región, propiedad de conteo (magnitud, profundidad, significancia), casillas(bins) del histograma.
Salidas	Histograma basado en el año, propiedad y región escogidas por el usuario.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
-------	-------------

Crear lista vacía	$O(1)$
Conseguir valores de árbol de fechas	$O(\log N)$
Insertar valores de fechas en lista	$O(\log N)$
Organización de lista	$O(\log N)$
Creación títulos para tabulate	$O(1)$
Código para tabulate	$O(\log N)$
TOTAL	$O(\log N)$

Pruebas Realizadas

Descripción de las pruebas de tiempos de ejecución y memoria utilizada. Incluir descripción del procedimiento, las condiciones, las herramientas y recursos utilizados (librerías, computadores donde se ejecutan las pruebas, entre otros).

Procesadores	Intel® Core™ i3-1011U CPU @ 2.10 GHz 2.59 GHz
Memoria RAM	8 GB
Sistema Operativo	Windows 10 Home – 64 bits

Entrada	Tiempo (ms)
small	4070
5 pct	4393
10 pct	4823
20 pct	6506
30 pct	6964
50 pct	9456
80 pct	10086
large	10457

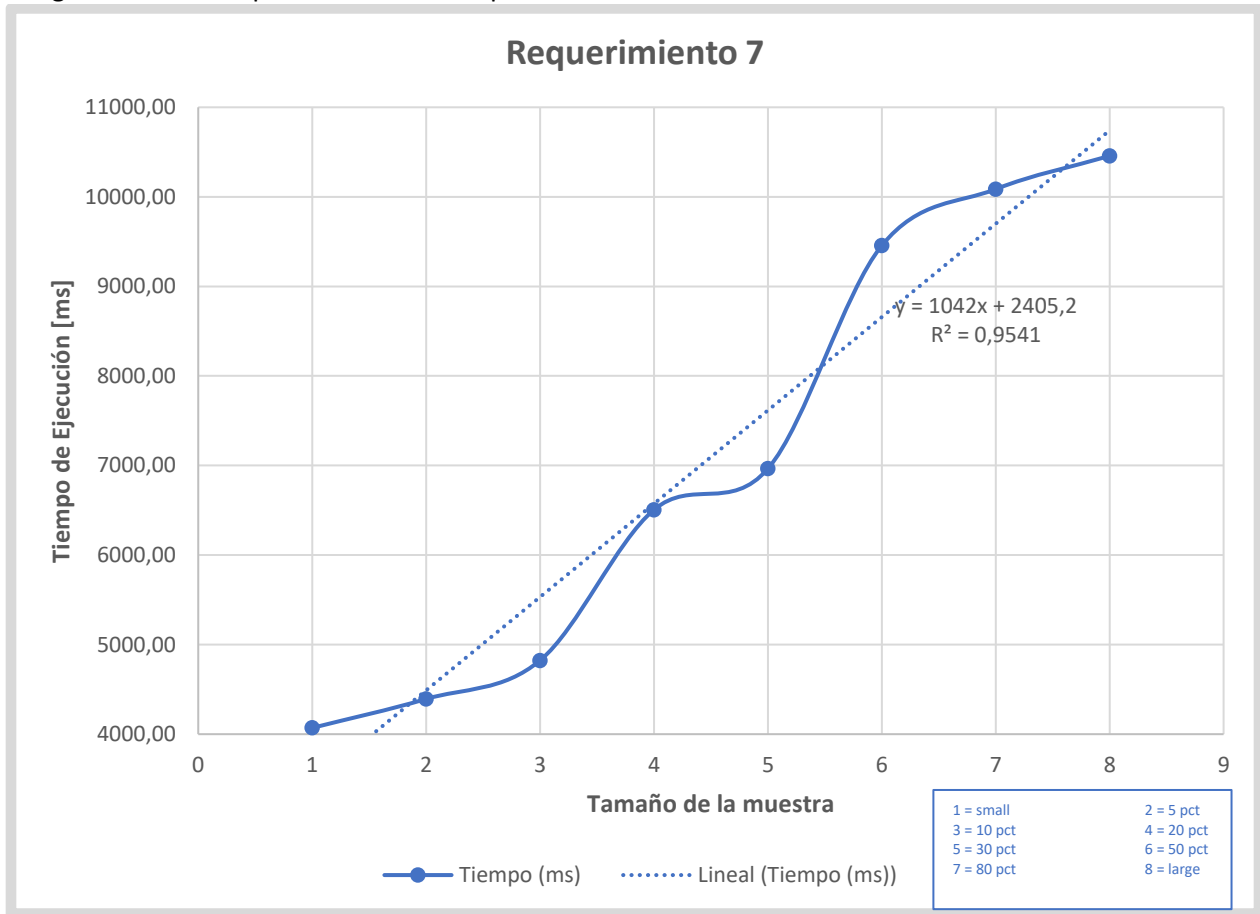
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	4070
5 pct	Dato2	4393
10 pct	Dato3	4823
20 pct	Dato4	6506
30 pct	Dato5	6964
50 pct	Dato6	9456
80 pct	Dato7	10086
large	Dato8	10457

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Aunque esta gráfica parece estar entre $O(N)$ y $O(\log N)$, decidimos de manera grupal que parece estar más cerca a $O(\log N)$. Esto se debe a que los valores en la tabla de datos no parecen haber pasado por ningún salto repentino en términos del tiempo de carga; en otras palabras, se puede ver que en las últimas cargas (desde el 50% hasta 'large') el tiempo de carga cambió ligeramente (sólo alrededor de 1000 milisegundos).